CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY & RESEARCH

Department of Information Technology

Subject Name: Data Engineering
Semester: 5th
Subject Code: ITUC301
Academic Year: 2026-27 (ODD)

PRACTICAL CIE - FILE

PRACTICAL – CIE-1

Problem Definition

An e-commerce organization collects data from multiple sources, including customer and product information in CSV format, transaction data in JSON format, and web application user activity logs generated continuously.

The objective is to design and implement a simplified end-to-end data engineering pipeline using Python and local files.

1. DATA ENGINEERING LIFECYCLE

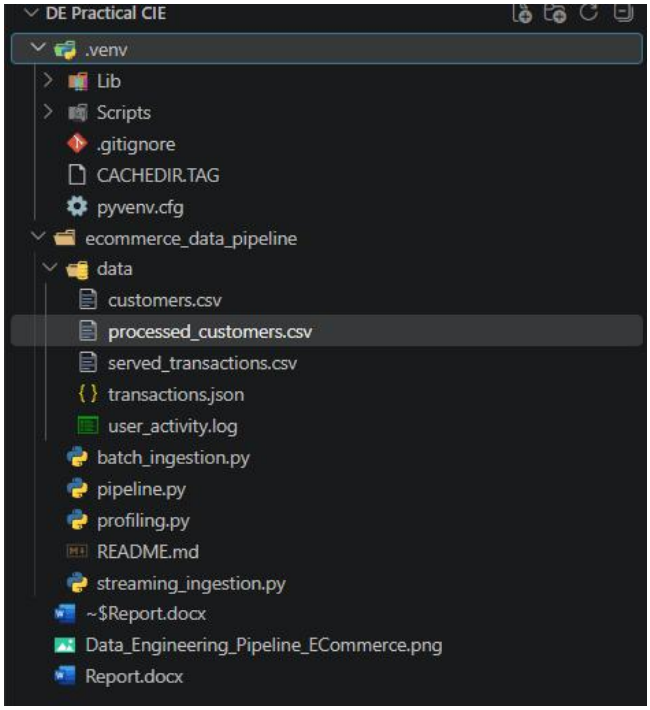
Objective

To design and demonstrate the basic data engineering lifecycle for the given e-commerce system.

Pipeline

Data Generation → Data Ingestion → Storage → Transformation → Data Serving

Folder Structure



Pipeline Diagram:

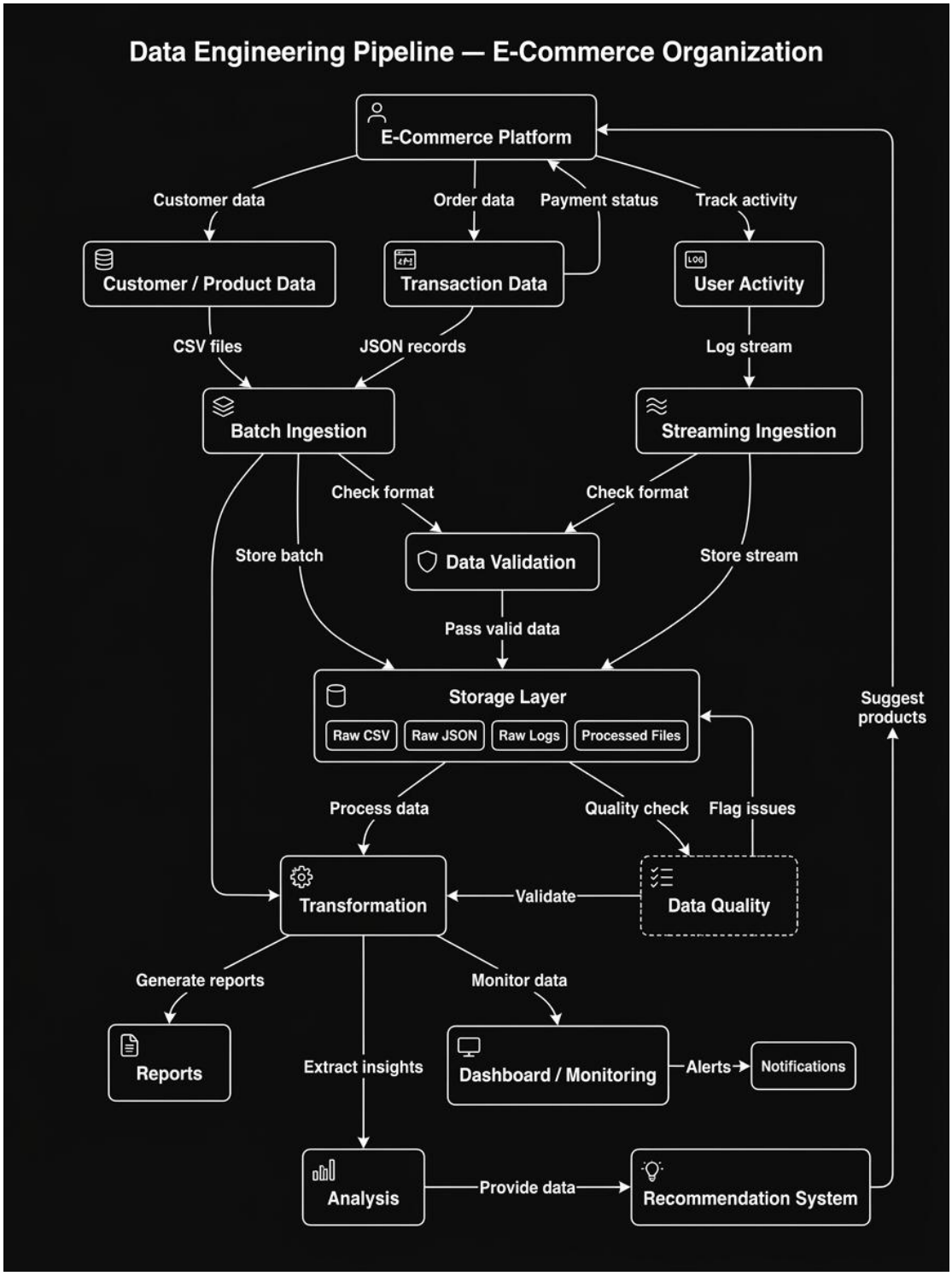


Figure 1: Data Engineering Lifecycle

Data Quality Consideration

Data quality should be checked during the pipeline to identify missing values, duplicate records, and inconsistent data formats before the data is used for analysis.

2. DATA GENERATION AND PROFILING

Objective

To create sample data from multiple sources and identify basic data quality issues.

Data Sources

1. Customer/Product Data

File: customers.csv

Fields:

customer_id, name, email, product, price

2. Transaction Data

File: transactions.json

Fields:

transaction_id, customer_id, amount, timestamp

3. User Activity Data

File: user_activity.log

Format:

timestamp, customer_id, event

These three sources represent CSV, JSON/API, and continuously generated log data.

Data Quality Issues

The sample data contains the following issues:

- Missing/null values
- Duplicate records
- Inconsistent timestamp formats

Profiling

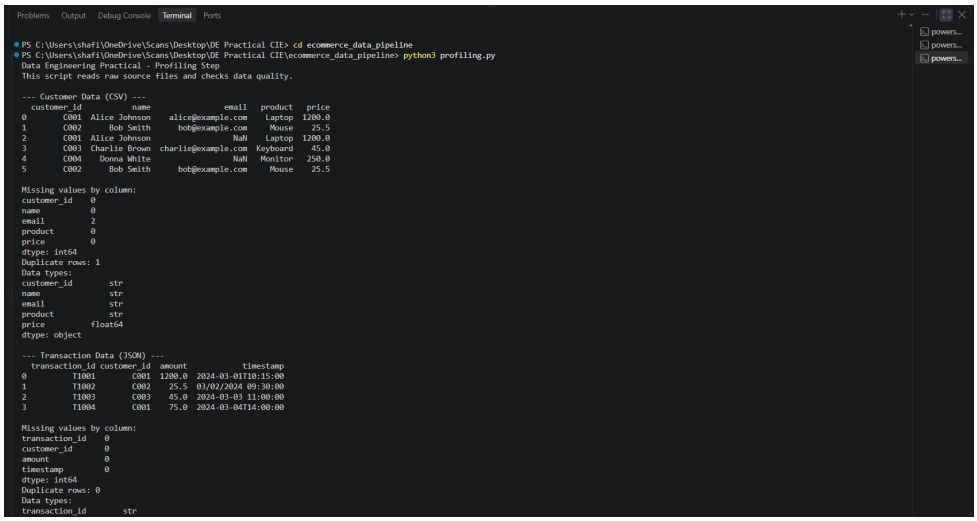
The Python profiling program performs:

1. Reads CSV data.
2. Reads JSON data.
3. Reads log data.
4. Displays records.
5. Checks missing values.
6. Checks duplicate records.
7. Displays data types.
8. Checks timestamp formats.

Observed Issues

- Missing email value
- Duplicate customer record
- Missing transaction amount
- Duplicate transaction record
- Inconsistent timestamp format

Figure 2: Data Profiling Output





3. BATCH AND STREAMING INGESTION

Objective

To implement and compare batch ingestion and streaming-style ingestion using Python.

A. Batch Ingestion

Batch ingestion processes a group of records together.

In this practical, customers.csv is used as the batch data source.

Process

customers.csv → Read Data → Remove Duplicates → Handle Missing Values → Save Processed Data

The processed data is stored as: data/processed_customers.csv

Steps

1. Read the CSV file.
2. Display raw records.
3. Remove duplicate records.
4. Handle missing email values.
5. Save cleaned data.
6. Display processed records.

Figure 3: Batch Ingestion Output

```
PS C:\Users\shafi\OneDrive\Scans\Desktop\DE Practical CIE\ecommerce_data_pipeline> python batch_ingestion.py

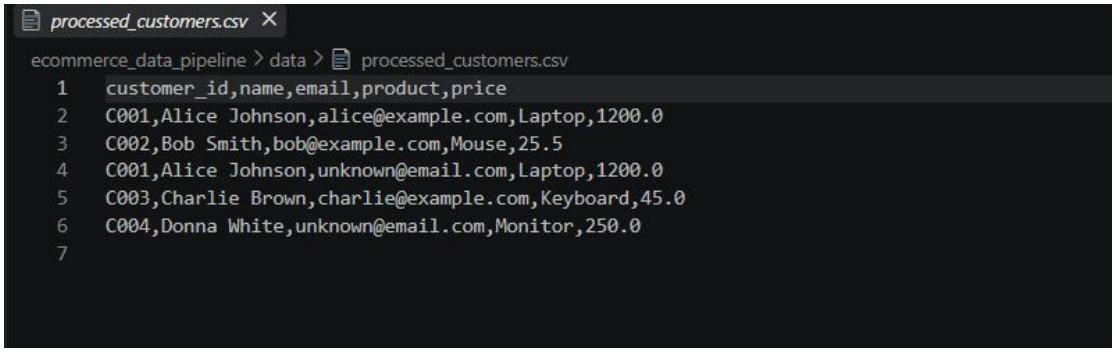
--- Batch Ingestion ---
Reading raw customer data from customers.csv
Original data:
  customer_id          name               email   product   price
0        C001  Alice Johnson    alice@example.com    Laptop  1200.0
1        C002      Bob Smith      bob@example.com     Mouse    25.5
2        C001  Alice Johnson                 NaN    Laptop  1200.0
3        C003  Charlie Brown  charlie@example.com  Keyboard    45.0
4        C004    Donna White                 NaN   Monitor   250.0
5        C002      Bob Smith      bob@example.com     Mouse    25.5

Removing duplicate records...
Handling missing email values...
Saving cleaned data to data/processed_customers.csv

Cleaned data:
  customer_id          name               email   product   price
0        C001  Alice Johnson    alice@example.com    Laptop  1200.0
1        C002      Bob Smith      bob@example.com     Mouse    25.5
2        C001  Alice Johnson    unknown@email.com    Laptop  1200.0
3        C003  Charlie Brown  charlie@example.com  Keyboard    45.0
4        C004    Donna White    unknown@email.com   Monitor   250.0

Batch ingestion completed.
PS C:\Users\shafi\OneDrive\Scans\Desktop\DE Practical CIE\ecommerce_data_pipeline>
```

Figure 4: Processed Customer Data



B. Streaming-Style Ingestion

Streaming ingestion processes incoming events individually.

In this practical, user_activity.log is used to simulate continuous web activity.

Process

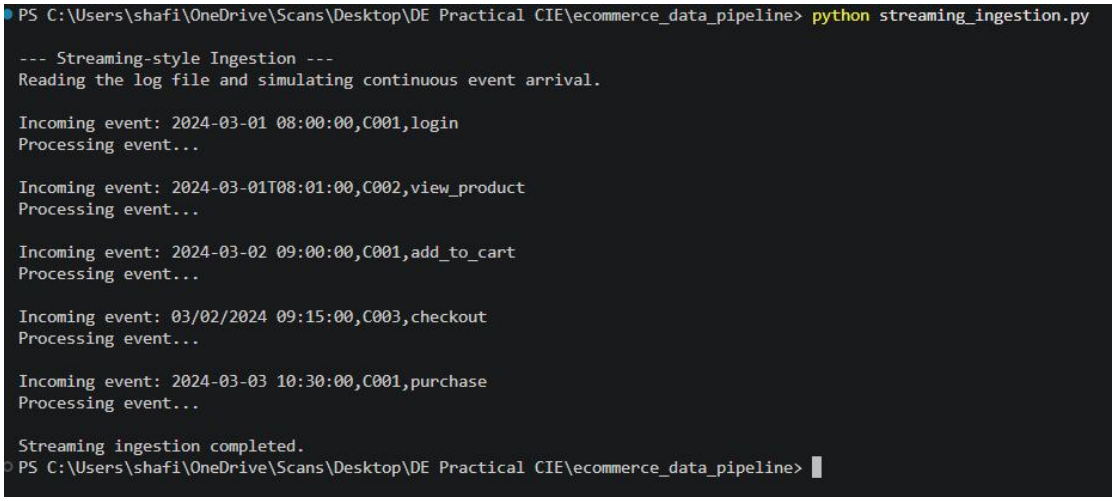
User Activity Log → Read Event → Process Event → Read Next Event → Process Event

The program uses time.sleep(1) to simulate continuous event arrival.

Steps

1. Read the log file.
2. Read one event at a time.
3. Display the incoming event.
4. Process the event.
5. Wait for a short interval.
6. Process the next event.

Figure 5: Streaming-Style Ingestion Output



Batch vs Streaming Comparison

Parameter	Batch Ingestion	Streaming Ingestion
Processing	Records in groups	Events individually
Latency	Higher	Lower
Data Arrival	Periodic/collected	Continuous
Throughput	High for large batches	Suitable for continuous events
Example	Historical transactions	User activity
Use Case	Reports and analysis	Real-time monitoring

Processing Latency

Batch ingestion has higher latency because data is collected before processing. Streaming ingestion has lower latency because events are processed as they arrive.

Throughput

Batch ingestion is suitable for processing large amounts of data together, while streaming ingestion is suitable for continuously arriving events.

Suitable Use Cases

Batch:

- Historical transactions
- Periodic reports
- Data analysis

Streaming:

- User activity monitoring
- Real-time event tracking
- Continuous monitoring

4. RESULT AND ANALYSIS

Result

The practical successfully demonstrates a simplified end-to-end data engineering pipeline using Python and local files.

The following operations were performed:

- CSV, JSON, and log data generation
- Data profiling
- Missing value detection

- Duplicate detection
- Format checking
- Batch ingestion
- Streaming-style ingestion
- Data transformation
- Processed data storage
- Data serving

The final serving output provides:

- Total transactions
- Total transaction amount
- Number of customers
- Number of user activity events

Figure 6: Final Data Serving Output

```
PS C:\Users\shafi\OneDrive\Scans\Desktop\DE Practical CIE\ecommerce_data_pipeline> python pipeline.py
E-commerce Data Engineering Pipeline
Data Generation -> Data Ingestion -> Storage -> Transformation -> Data Serving
Stage 1: Data Generation
Customer CSV, transaction JSON, and activity log files are available in the data folder.

Stage 2: Data Ingestion
Loaded 6 customer records
Loaded 4 transaction records
Loaded 5 activity log events

Stage 3: Storage
Data is stored locally in the data folder.

Stage 4: Transformation
Removed duplicates and filled missing emails.
  customer_id           name               email   product   price
0        C001  Alice Johnson    alice@example.com    Laptop  1200.0
1        C002      Bob Smith      bob@example.com     Mouse    25.5
2        C001  Alice Johnson    unknown@email.com    Laptop  1200.0
3        C003  Charlie Brown  charlie@example.com  Keyboard    45.0
4        C004    Donna White    unknown@email.com   Monitor   250.0

Stage 5: Data Serving
                         metric   value
0             total_transactions     4.0
1        total_transaction_amount  1345.5
2             number_of_customers     4.0
3  number_of_user_activity_events     5.0
Serving file saved to data/served_transactions.csv

Pipeline demonstration completed.
PS C:\Users\shafi\OneDrive\Scans\Desktop\DE Practical CIE\ecommerce_data_pipeline>
```

Analysis

Historical Transaction Processing:
Batch ingestion is more suitable because historical records can be collected and processed together efficiently.

Real-Time User Activity Monitoring:
Streaming ingestion is more suitable because events can be processed continuously with lower latency.

CONCLUSION:

The practical successfully demonstrates the basic data engineering lifecycle:

Data Generation → Data Ingestion → Storage → Transformation → Data Serving

CSV, JSON, and log files were used as different data sources. Basic profiling identified missing values, duplicate records, and inconsistent formats. Batch ingestion was used for historical data, while streaming-style ingestion was used for continuous user activity.

Therefore, batch ingestion is suitable for historical transaction processing, while streaming ingestion is suitable for real-time user activity monitoring.

GITHUB REPOSITORY LINK:

https://github.com/ShafinNigamana/D25DIT094-DE-Practicals/tree/main/DE%20Practical%20CIE